

PHASE 2 · SHORT VERSION

Reproducing EfficientNet-B0 for malaria detection

From the paper's own code to all 10 folds: results, findings and what we test next

AJ

J Amathziah

230139 · Kaggle runs,
repository

MM

C Murali Madhav

230115 · Code: notebook,
scripts

RY

Ravi Yadav

230131 · Slides

PJ

Pugazhendhi J

230066 · Report

What we did after Phase 1, and what we found



Phase 1 • reviewed 11 studies

Accuracy on the NIH cells is reported at 96–99.9 %, but most studies split images at random, so one patient's cells sit in both training and testing.



Phase 2 • reproduced the base paper

Marques et al. (2022): EfficientNet-B0 + a 10-fold ensemble. We rebuilt it from the authors' own code and trained all 10 folds.



Next • test it fairly

H1: does accuracy drop once slide-level leakage is removed? H2: does YUV stain normalisation win it back?

97.55 %

mean fold accuracy

paper 97.56 % · Δ -0.01 pp

97.62 %

10-model ensemble

paper 98.29 % · Δ -0.67 pp

99.73 %

ensemble ROC-AUC

paper 99.76 % · Δ -0.03 pp

≈ 11 min

per fold on an RTX 4090

paper ≈ 4 h 46 min per fold

All 10 folds within 1 percentage point of the paper. Every metric is computed from the saved predictions in `results/A0_10fold_paper/`.

Why EfficientNet-B0 with a 10-fold ensemble



Open, executable code

The authors published their executed Jupyter notebook as a supplementary file, so a faithful rebuild is possible.



Strict validation

Stratified 10-fold cross-validation plus a separate hold-out set that no model trains on.



Light backbone

4.2 M parameters in total: the smallest EfficientNet, close to our deployability goal.



Strong, not at the ceiling

98.29 % leaves room to see whether a change helps or hurts.

THE PAPER'S TARGETS (from its own printed results)

97.56 %

mean fold accuracy (Table 5)

97.69 %

single model on hold-out (Table 6)

98.29 %

ensemble accuracy

99.76 %

ensemble ROC-AUC

The authors' notebook differs from the paper's text

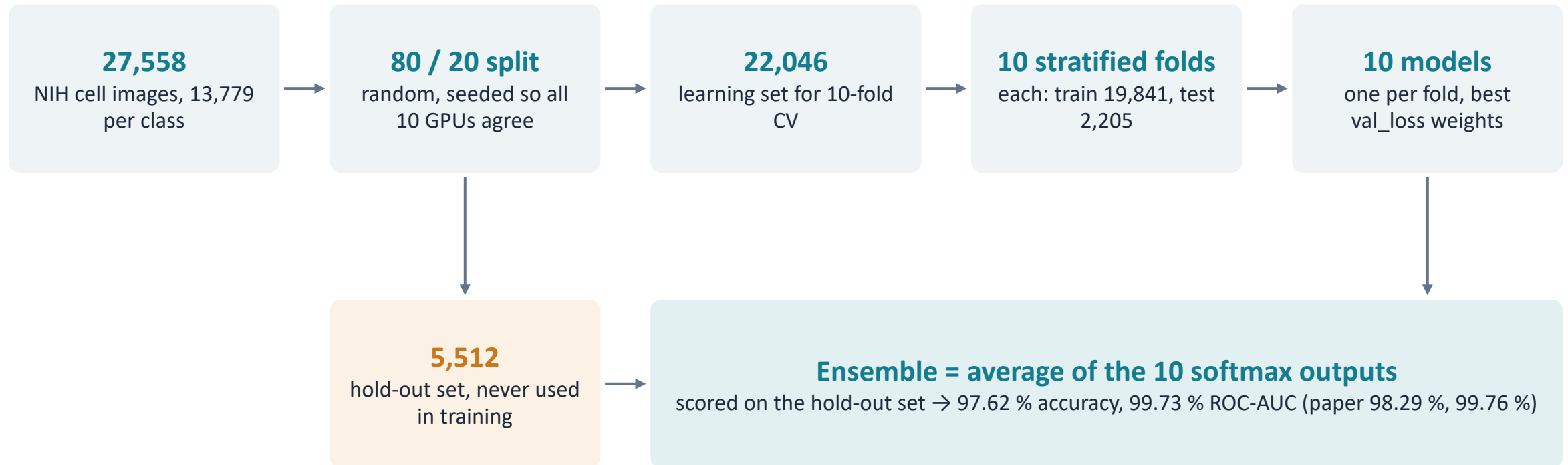
Setting	Paper's code (used now)	Our first reproduction (from the text)
Pretrained weights	noisy-student · efficientnet 1.1.1 (qubvel)	ImageNet · tf.keras.applications
Classifier head	GAP → 128 → 64 → 32 → 2 softmax, dropout 0.3	GAP → dropout 0.2 → 1 sigmoid
Loss	categorical cross-entropy, label smoothing 0.1	binary cross-entropy
Schedule	33 epochs, LR × 0.5 on plateau, keep best val_loss	≤ 15 epochs, LR × 0.1, early stopping
Batch size	16 (1,240 steps per epoch)	128 (64 per GPU, 2 GPUs)
Augmentation	full albumentations pipeline (noise, blur, distortions, CLAHE...)	flips + 90° rotations

Source: Springer supplementary file MOESM1, the authors' executed notebook (30 Jan 2021). Our notebook now follows it line by line; 32 automated tests check the recipe.



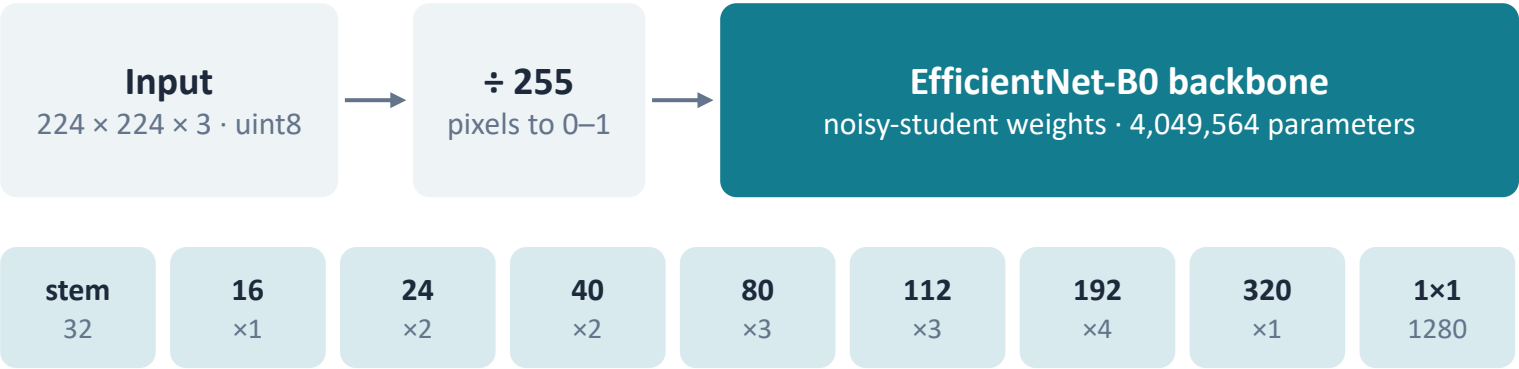
Two quirks in the paper itself: its code evaluates on augmented images, and its text swaps precision and recall (true recall 97.74 %, precision 98.82 %).

One split, ten fold models, one ensemble



Every fold model is also scored on its own CV test part (the paper's Table 5) and on the hold-out set (Table 6). Each fold stores a fingerprint of the split (a0cc112e...): all 10 folds match.

EfficientNet-B0 with the paper's dense head



Inside the backbone: a stem convolution, 16 MBConv blocks in 7 stages (output channels, repeats), and a 1×1 convolution $\rightarrow 7 \times 7 \times 1280$ features.



Output: P(parasitized), P(uninfected). Loss: categorical cross-entropy with label smoothing 0.1.

Part	Parameters
EfficientNet-B0 backbone	4,049,564
Dense head	174,370
Total	4,223,934
Trainable	4,181,918
Non-trainable (BatchNorm)	42,016

Identical to the paper's Table 2, digit for digit, checked by an automated test.

Noisy-student weights: EfficientNet-B0 pre-trained on ImageNet plus self-training on unlabelled images (Xie et al., 2020). The paper loads them from the efficientnet package; so do we.

How to read the metrics

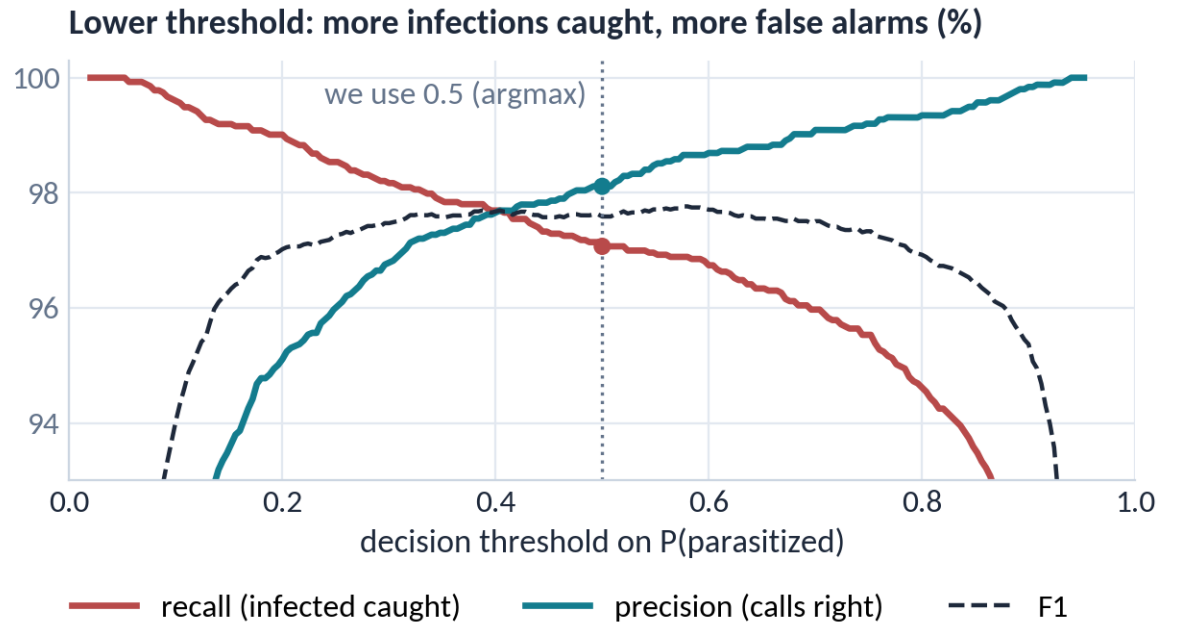
	model says PARASITIZED	model says UNINFECTED
truth: parasitized (2,730)	TP = 2,650 infected, caught	FN = 80 infected, missed
truth: uninfected (2,782)	FP = 51 healthy, false alarm	TN = 2,731 healthy, cleared

Accuracy = $(TP + TN) / \text{all} = 5,381 / 5,512 = 97.62 \%$

Recall (sensitivity) = $TP / (TP + FN)$: infected cells caught · 97.07 %

Precision = $TP / (TP + FP)$: 'parasitized' calls that are right · 98.11 %

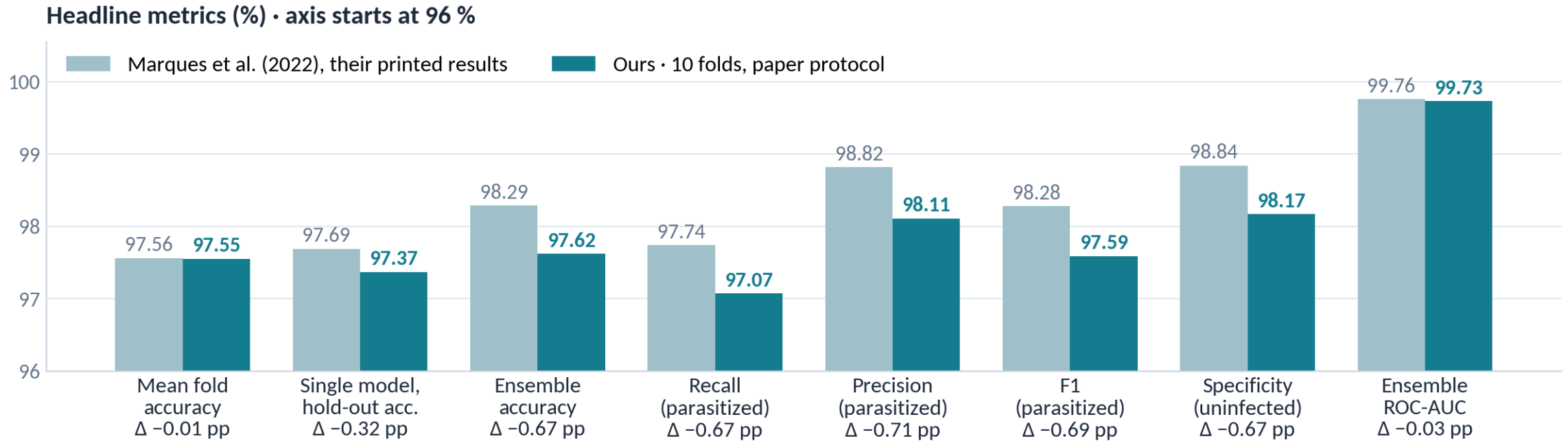
F1 = harmonic mean of precision and recall · 97.59 %



ROC-AUC: chance that an infected cell outscores a healthy one · 99.73 %

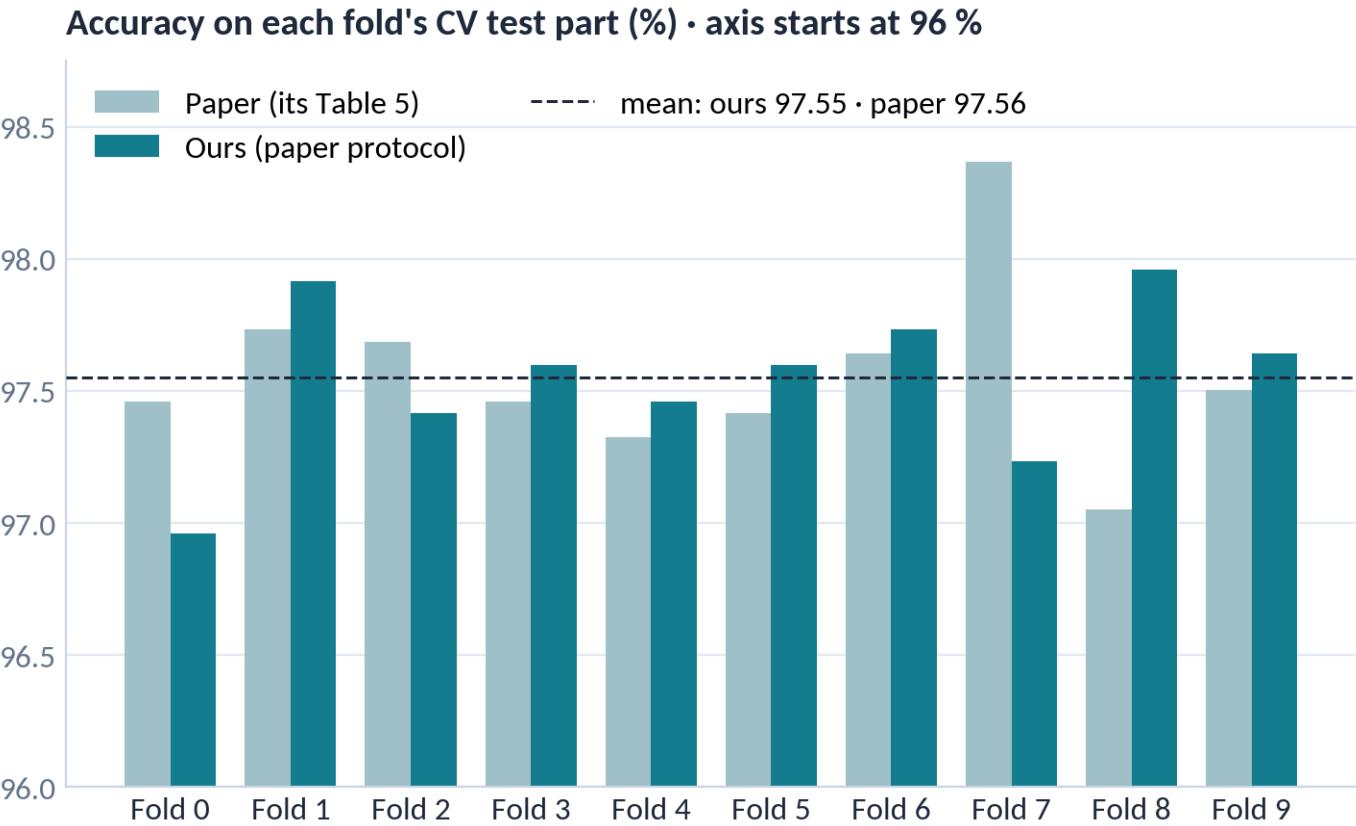
pp (percentage point) = plain difference of two percentages: $97.62 - 98.29 = -0.67 \text{ pp}$

Paper vs. ours on the 5,512-image hold-out set



All within our 1 pp success criterion; mean fold accuracy matches to 0.01 pp. On clean (un-augmented) images the ensemble scores 97.64 %, so the paper's augmented evaluation changes little.

Ten consistent folds, and where the ensemble errs



Ensemble on 5,512 hold-out cells

	Pred. parasitized	Pred. uninfected
Parasitized	2,650	80
Uninfected	51	2,731

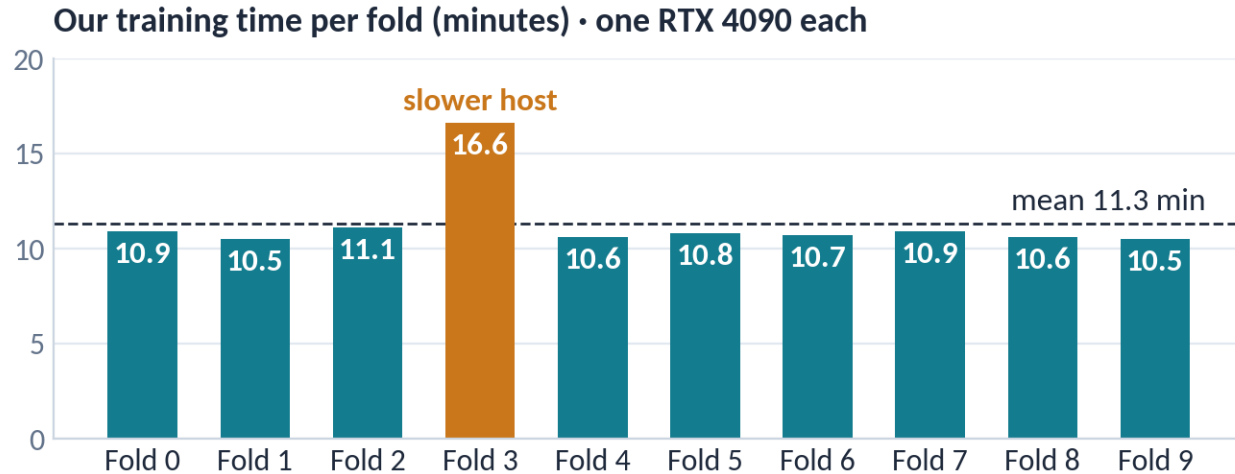
Paper: 62 missed, 32 flagged. Recall (catching infected cells) is our priority metric.

Stable training, near-perfect ranking



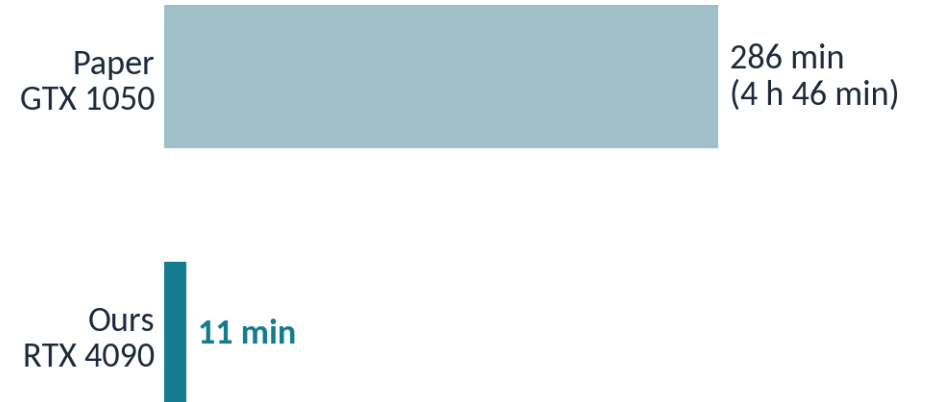
Validation accuracy settles near 97.5 % after about 10 epochs; the learning rate halves on plateaus and the lowest-val_loss weights are kept. The ensemble's ROC hugs the top-left corner: AUC 99.73 % (paper 99.76 %).

Ten GPUs in parallel: about 11 minutes per fold



10 RunPod pods (Secure Cloud, RTX 4090, 16 vCPU each), one fold per pod, all training at the same time. Fold 3 landed on a slower host (16.6 min).

Training time per fold: $\approx 25\times$ faster



$\approx 25\times$
faster per fold

≈ 19 s
per epoch (median)

1 d 23 h
the paper's full run



The 10 trained models are published as a GitHub release, so nobody has to retrain to use or evaluate them.

Two proposed changes, one variable at a time



H1 · VALIDATION

Leakage-free, slide-grouped split

CHANGE

Split by slide/patient (StratifiedGroupKFold) instead of random images. Model unchanged.

WE EXPECT

Accuracy and MCC fall below the image-split baseline, and fold-to-fold spread widens.



H2 · PREPROCESSING

YUV stain normalisation (Umer [4])

CHANGE

RGB → YUV → histogram-equalise Y → RGB before EfficientNet-B0. Grouped split kept fixed.

WE EXPECT

Higher recall and MCC than the H1 model on unseen slides; little change on the random split.

ID	Split	Preprocessing	Answers	Status
A0	image (paper)	none	baseline: does it match the paper?	done ✓ (10 folds)
A1	slide-grouped	none	H1: leakage gap (A0 – A1)	next
A2	slide-grouped	YUV + hist-eq	H2: gain on unseen slides (A2 – A1)	next
A3	image (paper)	YUV + hist-eq	control: is the gain slide-specific?	next

CONCLUSION

Where we stand



Reproduced

The paper's recipe, rebuilt from its own code, lands within 1 pp on every metric (mean fold 97.55 vs 97.56 %).



Corrected

The paper evaluates on augmented images and swaps precision and recall in its text; we report both views.



Fast and shareable

10 folds in parallel, $\approx$ 11 min each instead of $\approx$ 4 h 46 min; the trained models are published.



Next

A1–A3 test whether 97–98 % holds on unseen slides (H1) and whether stain normalisation helps (H2).

Thank you · questions welcome

github.com/amathziah/malaria-detection